# Hệ thống phát hiện xâm nhập phân tán thời gian thực trên cụm thiết bị biên Jetson Orin Nano Super với Kafka và PySpark

**Nguyễn Vũ Thái**[1] Bùi Văn Dũng[2] Đỗ Trí Nhựt[2] Nguyễn Văn Dũ[3]

SOICT 2026

[1]Phân hiệu tại TP. Hồ Chí Minh, Trường Đại học Giao thông Vận tải, TP. Hồ Chí Minh, Việt Nam
[2]Trường Đại học Công nghệ Thông tin, ĐHQG-HCM, TP. Hồ Chí Minh, Việt Nam
[3]Khoa Công nghệ Thông tin, Trường Đại học Nông Lâm, TP. Hồ Chí Minh, Việt Nam

# I. VẤN ĐỀ VÀ ĐÓNG GÓP

# Phát hiện xâm nhập ngay tại thiết bị biên: còn ba khoảng trống

Mô hình phát hiện xâm nhập nay đã *chạy được* trên bo mạch nhỏ, nhưng nếu một bo phải xét **mọi** luồng thì nó quá tải rất nhanh.

**Nghiên cứu trước còn thiếu**

1. Chỉ đo trên **một thiết bị**, không chia việc cho nhiều bo.
2. Hiếm khi báo **cùng lúc** tốc độ, độ trễ và năng lượng.
3. Điểm số offline lấy từ quy trình **rò rỉ dữ liệu**.

**Câu hỏi**

Có hai bo Jetson thì nên *đặt* việc phát hiện lên chúng thế nào?

**Cách làm**

Tách làm hai: **cổng lọc** rẻ trên bo #1, **bộ phân loại** nặng trên bo #2, nối bằng Kafka.

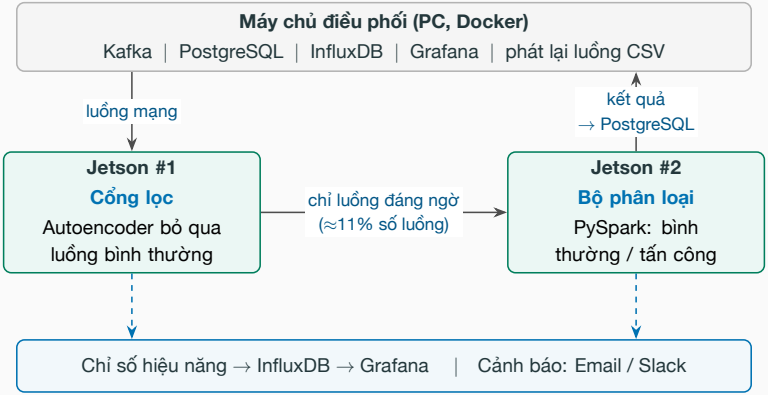
# Các đóng góp

Các thành phần đều quen thuộc (autoencoder, Kafka, Spark). **Không đề xuất mô hình mới**: đóng góp là những gì đo được trên phần cứng biên thật.

1. **So sánh cách đặt hai tầng** lên cụm hai Jetson: một bo, tách tầng trên một bo và trên hai bo, nhân bản ngang, cụm Spark.
2. **Đo đầu cuối**: mức tải chịu được, độ trễ, năng lượng mỗi dự đoán *và* precision / recall / F1 của cả pipeline sau cổng lọc, trên CICIDS2017 và CSE-CIC-IDS2018.
3. So sánh **cùng một mô hình** trên PySpark, NumPy và ONNX Runtime.
4. Mã nguồn, cấu hình và kịch bản đo **công khai**, chạy lại được.

# II. HỆ THỐNG VÀ CÁCH ĐO

# Kiến trúc: tách cổng lọc và bộ phân loại ra hai bo




Chỉ các luồng đáng ngờ mới đi qua Kafka sang bo #2.

# Hai tầng làm việc thế nào

## Tầng 1 — cổng lọc (autoencoder)

- Học cách *tái tạo* lưu lượng bình thường.
- Luồng tái tạo tốt (sai số nhỏ) → coi là bình thường, **dừng luôn**.
- Luồng tái tạo kém → đáng ngờ, chuyển sang tầng 2.
- Ngưỡng sai số chọn *trước* trên 10% dữ liệu bình thường, không nhìn tập kiểm tra.

## Tầng 2 — bộ phân loại (PySpark)

- Random Forest trên **30 đặc trưng** được SHAP chọn (bài FAIR), không có cổng đích, F1 0,9959.
- Chỉ phải xét số ít luồng mà cổng chuyển sang.

# Vì sao bộ phân loại là Random Forest trên Spark

- **Random Forest có sẵn trong Spark ML**, nên mô hình xuất ra chạy được trên Jetson (ARM64) mà không cần thư viện native như XGBoost.
- **Ít đặc trưng** (30 thay vì 77) thì dựng vector và suy luận nhanh hơn trên bo 8 GB.
- **Cùng một bộ công cụ** (Spark) vừa huấn luyện trên cụm vừa chạy trên thiết bị — cái giá của lựa chọn này ở Kết quả 7.

# Các cách triển khai và cách đo

**Một bo** làm hết mọi việc, tắt hoặc bật cổng lọc.

**A: Tách tầng** cổng trên #1, phân loại trên #2.

**A1: Tách, một bo** cổng và phân loại là hai tiến trình trên #2.

**B: Nhân bản** mỗi bo làm đủ cả hai việc (tắt cổng).

**C: Cụm Spark** PC làm master, hai bo làm worker.

Theo dõi **từng luồng từ lúc gửi tới lúc có kết quả**, tính cả thời gian chờ. **Chịu được**: mức tải cao nhất mà mọi luồng có kết quả trong vòng 10 giây sau khi ngừng tải.

## Thiết lập đo

2× Jetson Orin Nano Super: CPU 6 nhân, 8 GB RAM

PySpark 3.4.1, Kafka 3.5, mạng Wi-Fi

10.000 luồng từ tập kiểm tra, 15% tấn công

+ 10.000 luồng CSE-CIC-IDS2018

Quét mức tải 1–40 luồng/giây, mỗi mức 2 lần

Năng lượng đo bằng `tegrastats`

# III. KẾT QUẢ VÀ KẾT LUẬN

# Kết quả 1: tách hai tầng chịu được tải gấp 7–9 lần

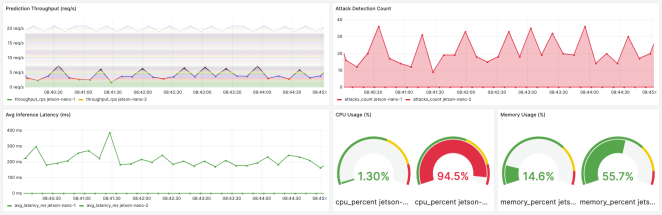
| Cách triển khai | Chịu được (luồng/giây) | p95 (giây) |
|---|---|---|
| Một bo | 2,5 | 11,3 |
| Một bo + cổng | ≈2,5 | 7,8 |
| B: Nhân bản | 3,3 | 8,2 |
| C: Cụm Spark | 9–17 | 5,2 |
| A1: Tách, một bo | **17,8** | 5,3 |
| **A: Tách tầng** | **22,4** | **7,1** |

**p95**: 95% số luồng có kết quả nhanh hơn mức này, tính từ lúc gửi, gồm cả thời gian chờ.

- **Cổng chạy chung tiến trình với bộ phân loại: không nhanh hơn.**
- **Tách thành hai tiến trình trên cùng một bo: gấp 7 lần.**
- Thêm bo thứ hai chỉ tăng thêm khoảng một phần tư (22,4).

**Lợi ích đến từ việc tách hai tầng**, không phải từ bo thứ hai.

# Kết quả 2: nhìn trực tiếp thấy bo nào đang gánh việc




Lúc tải cao nhất, chế độ tách tầng

(phát lại đủ 10.000 luồng, mô hình đang dùng):

- bo cổng lọc: **1,3% CPU**
- bo phân loại: **94,5% CPU**

Bộ phân loại chiếm gần hết sức tính — nên tách nó ra khỏi đường nhận dữ liệu là đáng.

# Kết quả 3: năng lượng mỗi dự đoán giảm 5 lần

| Cách triển khai | Mức tải | J / dự đoán |
|---|---|---|
| Một bo | 2,0/giây | 3,04 |
| Một bo + cổng | 2,0/giây | 3,04 |
| B: Nhân bản | 3,9/giây | 3,20 |
| C: Cụm Spark | 9,5/giây | 1,15 |
| A1: Tách, một bo | 17,8/giây | **0,32** |
| **A: Tách tầng** | 18,4/giây | **0,60** |

- Đo ở đúng mức tải mà mỗi chế độ chịu được.
- J mỗi dự đoán = công suất của các bo chia cho số dự đoán mỗi giây.
- Tách tầng dùng hai bo (11 W) nhưng ra nhiều dự đoán hơn hẳn, nên mỗi dự đoán tốn ít hơn.
- Tách trên một bo còn rẻ hơn (0,32 J): không tốn điện nền của bo thứ hai.

Tách tầng: **3,0 J** → **0,60 J** mỗi dự đoán (hai bo), **0,32 J** (một bo).

# Kết quả 4: cổng lọc cho lọt bao nhiêu tấn công

| Ngưỡng cổng (phân vị) | Tấn công chuyển tiếp | Báo nhầm | Luồng bỏ qua |
|---|---|---|---|
| 0,99 | 72,4% | 0,99% | 88,3% |
| **0,995 (đang dùng)** | **68,1%** | **0,49%** | 89,4% |
| 0,999 | 39,8% | 0,09% | 94,0% |

Offline trên tập kiểm tra, cổng dùng đủ 77 đặc trưng. Chạy thật: xem Kết quả 5.

- Luồng bị cổng bỏ qua là *chốt luôn*: **khoảng 30% luồng tấn công** không tới được bộ phân loại.
- Nếu cổng chỉ dùng 30 đặc trưng của bộ phân loại thì chỉ chuyển tiếp được 44,8%.
- Người vận hành chọn điểm hợp với sức máy.

Tải tăng phải trả bằng recall: cổng là một **sự đánh đổi có thể chỉnh**, không phải bộ lọc miễn phí.

# Kết quả 5: đo đầu cuối cả pipeline sau cổng lọc

| Cách triển khai | Tấn công | Lọt: cổng / phân loại | Báo nhầm | F1 |
|---|---|---|---|---|
| **A: phát lại đủ** | 1.518 | 463 / 0 | 0 | **0,820** |
| A1: Tách, một bo | 882 | 278 / 0 | 0 | 0,813 |
| C: Cụm Spark | 381 | 115 / 0 | 0 | 0,822 |
| Tắt cổng | 242 | – / 0 | 0 | 1,000 |

Dự đoán cuối cùng của pipeline so với nhãn. Phát lại đủ: toàn bộ 10.000 luồng một lần, 20 luồng/giây trong 500 giây. Bộ phân loại một mình, offline: F1 0,996.

- **Precision 100%**: không báo nhầm luồng nào trong 8.483 luồng bình thường.
- **Recall 69,5%**: *mọi* tấn công bị lọt đều do cổng bỏ qua. Bộ phân loại không sai luồng nào nó nhận.
- F1 đầu cuối do ngưỡng cổng quyết định.

Phản biện hỏi đúng điều này: **P / R / F1 của cả pipeline**, không phải của bộ phân loại offline.

# Kết quả 6: sang mạng khác — CSE-CIC-IDS2018

| Luồng phát lại | CICIDS2017 | CSE-CIC-IDS2018 |
|---|---|---|
| Cổng chuyển sang | 11% | **30%** |
| Luồng bình thường bị chuyển | 0,6% | 28% |
| Chịu được (luồng/giây) | 22,4 | **≈8** |
| Recall / F1 | 69,5% / 0,82 | **0,8% / 0,016** |
| Bộ phân loại một mình, recall | 99,8% | 1,8% |

Cùng pipeline tách tầng, mô hình huấn luyện trên CICIDS2017, mỗi bộ 10.000 luồng phát lại một lần. Precision 100% ở cả hai.

- Cổng coi lưu lượng bình thường của mạng mới là bất thường: bộ phân loại phải nhận **gấp 3 lần**.
- Chính bộ phân loại cũng bỏ sót gần hết tấn công 2018.
- Offline: 1% nhãn của mạng mới đưa F1 lên lại 0,92.

Cả khả năng phát hiện *lẫn mức tải* đều không mang sang được: mạng mới cần **chỉnh lại ngưỡng cổng** và **huấn luyện lại**.

# Kết quả 7: cái giá của việc dùng Spark để suy luận

Đúng khu rừng đang triển khai, xuất sang các bộ máy chạy khác, cùng một bo:

| Bộ máy chạy | Luồng/giây | p95 mỗi lô (ms) | mJ mỗi luồng |
|---|---|---|---|
| PySpark (đang dùng) | 1,4 | 7515 | 1506 |
| NumPy | 54,8 | 194 | 11,3 |
| **ONNX Runtime** | **10.092** | **1,2** | **0,087** |

Lô 10 luồng, như khi triển khai. Ba bộ máy cho dự đoán giống nhau 100%. mJ mỗi luồng: năng lượng tăng thêm so với lúc bo nghỉ. Mỗi lần gọi Spark tốn $\approx$7 giây với mọi lô tới 1.000 luồng, nên ở lô 10.000 khoảng cách chỉ còn $\approx$27 lần (slide dự phòng).

ONNX nhanh gấp **khoảng 7.200 lần** PySpark. Hướng tiếp theo: **huấn luyện bằng Spark, suy luận bằng ONNX/TensorRT**.

# Kết luận

## Những gì đã chỉ ra

- Tách hai tầng: **2,5 → 17,8 luồng/giây** trên một bo (0,32 J), **22,4** trên hai bo (0,60 J), p95 ≈7 giây.
- Đầu cuối: **precision 100%**, **recall 69,5%**, F1 0,82. Mọi tấn công lọt đều do cổng.
- Sang CSE-CIC-IDS2018: recall **0,8%**, tải còn ≈8 luồng/giây.
- Spark là nút thắt suy luận: ONNX nhanh gấp khoảng 7.200 lần.

## Hạn chế

- Hai bo, mạng Wi-Fi, dữ liệu phát lại.
- Kafka và CSDL vẫn nằm trên máy chủ PC.
- GPU chưa dùng; chưa thử tấn công né tránh.

## Hướng tiếp theo

- Suy luận bằng ONNX/TensorRT (nhanh gấp ≈7.200 lần).
- Lưu lượng thật; huấn luyện lại khi dữ liệu trôi.

**Xin cảm ơn — kính mời câu hỏi**

# Tài liệu tham khảo chính


[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016. **điện toán biên**

[3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009. **cổng lọc**

[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP**

[5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark**

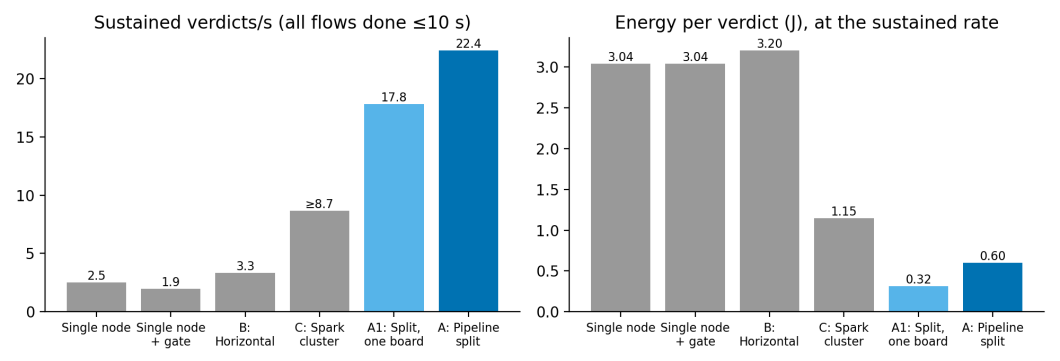
[6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**

[7] Kiran Kumar, K. M. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025. **hệ thống gần nhất**

[8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **chống rò rỉ**

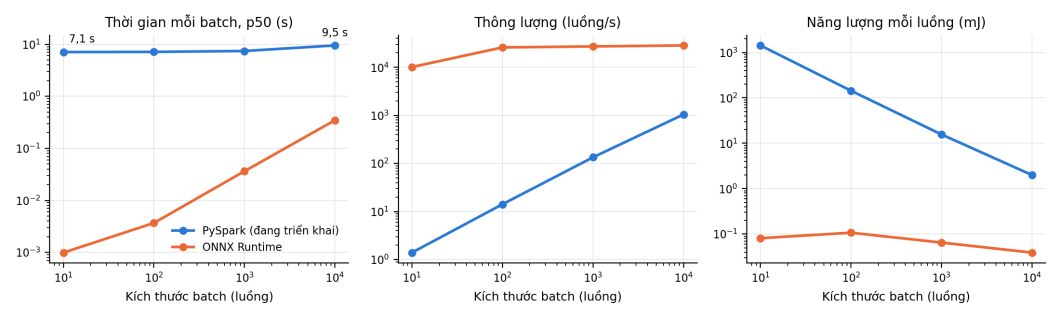
# Slide dự phòng

# Dự phòng: mức tải chịu được và năng lượng



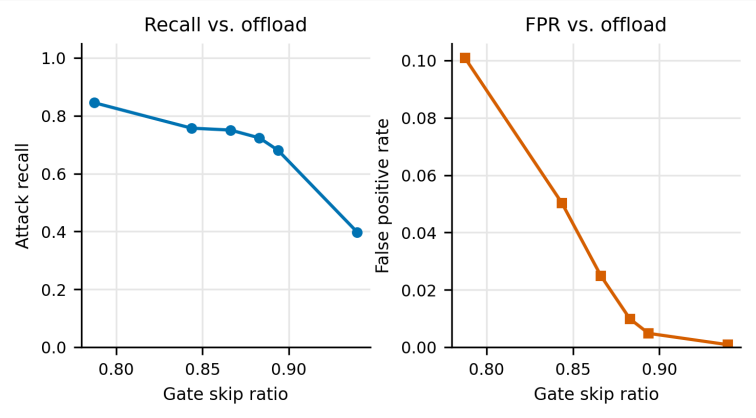

Số dự đoán mỗi giây chịu được, mọi luồng xong trong 10 giây (trái), năng lượng mỗi dự đoán ở mức tải đó (phải).

# Dự phòng: Spark tốn ở mỗi lần gọi, không phải ở các cây



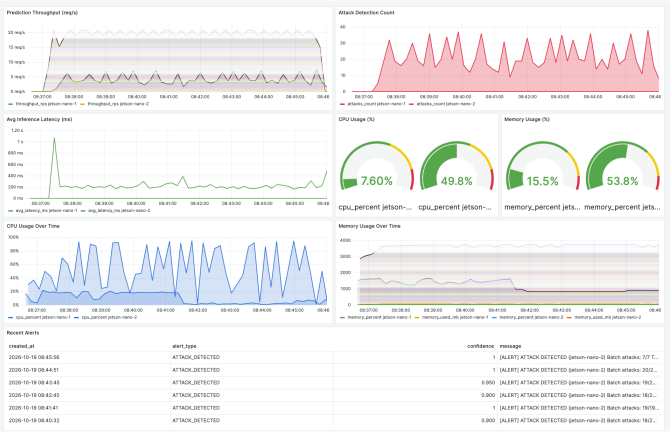

Cùng khu rừng, cùng bo, lô từ 10 tới 10.000 luồng. Một lần gọi PySpark mất 7,1–7,4 giây tới lô 1.000 và 9,5 giây ở lô 10.000. ONNX bão hòa quanh 28.000 luồng/giây. Khoảng cách giảm từ ≈7.200 lần (lô 10) xuống ≈27 lần (lô 10.000).

# Dự phòng: đường cong vận hành của cổng lọc




Recall tấn công (trái) và tỷ lệ báo nhầm (phải) theo tỷ lệ luồng bị bỏ qua. Bỏ qua nhiều hơn thì Spark nhẹ hơn nhưng lọt nhiều tấn công hơn.

# Dự phòng: bảng điều khiển giám sát đầy đủ




Tốc độ từng bo, số tấn công phát hiện, độ trễ suy luận, CPU/RAM hai bo, cảnh báo lưu trong PostgreSQL.

# Dự phòng: phụ thuộc vào máy chủ PC

- Kafka, PostgreSQL, InfluxDB, Grafana chạy trên máy chủ: luồng vào qua broker của nó, kết quả lưu ở đó. **Máy chủ hỏng thì ngừng phát hiện**, dù hai bo vẫn chạy tốt.
- Với PySpark, máy chủ *không* phải nút thắt: các bo bão hòa trước, độ trễ khứ hồi máy chủ–bo 4–9 ms qua Wi-Fi.
- Nhưng nó giới hạn việc mở rộng: thêm bo là thêm một consumer của cùng broker và một nơi ghi vào cùng CSDL (mỗi dự đoán một lệnh INSERT).
- Triển khai thực tế: broker đặt trên hoặc cạnh bo nhận luồng, kết quả đệm tại chỗ rồi gửi về theo lô, Kafka nhân bản giữa các bo. **Chưa đo.**

# Dự phòng: quy trình đo

- Máy chủ khởi động các vai trò trên Jetson qua SSH, tạo tải, thu mọi dự đoán.
- Mỗi dự đoán lưu thời điểm gửi và số dòng của luồng: độ trễ tính từ lúc gửi, *gồm cả thời gian chờ*. Tỷ lệ bỏ qua và tỷ lệ bắt tấn công tính từ chính các luồng.
- Quét mức tải cho từng chế độ. Một mức được coi là chịu được khi mọi luồng gửi trong 45 giây đều có kết quả trong vòng 10 giây sau đó.
- Mỗi lần chạy một consumer group Kafka riêng. Mỗi tầng xử lý lô 20 luồng hoặc sau 1 giây.
- Năng lượng: `tegrastats`, 30 giây nền rồi cửa sổ tải ở mức chịu được. Chế độ hai bo cộng cả hai bo.